# Kwantitatieve Biologie

Vakgroep Biologie, Universiteit Utrecht

2024-06-20

# Inhoudsopgave

# Deel I

# Kwantitatieve Biologie

# 1 Algemene inleiding cursus

Ik ben een algemene inleiding hoor.

# Deel II

# Introductie tot programmeren

Biologie in de 21e eeuw vereist analyse van grote hoeveelheden data, en modellering om complexe biologische systemen beter te begrijpen en te voorspellen. In dit onderdeel van de cursus maak je kort kennis met de basisprincipes van R, zodat ook jij straks voorspellingen kan doen, kan aantonen of een gemeten verschil statistisch significant is, en mooie grafieken kan maken.




Om kikkers te begrijpen kunnen we niet alleen heel hard naar kikkers kijken (afbeelding gegenereerd met Dall-E, juli 2024)

# 2 Dag 1: beginnen met Rstudio

Stap 1: R zelf installeren.

R kun je eenvoudig installeren via de CRAN website.

- Windows-gebruikers: ga naar "Download R for Windows", klikken op de link "base" en vervolgens op de link "Download R x.x.x for Windows" om het installatiebestand te downloaden. Open het gedownloade .exe bestand en volg de instructies om R te installeren.
- Mac-gebruikers: "Download R for (Mac) OS X". Download het juiste installatiebestand voor jouw macOS-versie. Let op: als je een oudere Mac gebruikt moet je de "Intel Mac" versie downloaden. Open het .pkg bestand en volg de instructies.

Stap 2: Rstudio installeren.

Nadat je R geïnstalleerd hebt, kun je beginnen met het installeren van Rstudio. Deze downloaden we vanaf de website van Rstudio. Deze website legt ook uit dat je wel eerst R moet installeren, maar dat hebben we al gedaan. We kunnen dus meteen naar stap 2: Install Rstudio. Deze website herkent vanzelf wel besturingssysteem je gebruikt, dus hoef je het alleen nog maar te installeren:

- Windows-gebruikers: volg de instructies van de installatie-wizard
- Mac-gebruiker: open de .dmg en sleep 'Rstudio' naar je Applications folder

Nu kun je Rstudio opstarten, en zou alles moeten werken[1]

[1] In een enkel geval kan Rstudio de installatie van R niet automatisch vinden. Ga dan naar 'Tools' > 'Global options', en dan naar het tabblad 'General'. Hier kan je handmatig instellen waar R geïnstalleerd staat. Vraag waar nodig hulp van je student-assistent (of ChatGPT ;])

## 2.1 Je eerste script

Nu je Rstudio hebt opgestart, gaan we eerst een nieuw script aanmaken. Klik hiervoor op de knop `New file` en kies `R script`. Nu opent zich een nieuw venster, met daar in een `script` zonder naam. Sla dit script op op een logische plek. Maar wat is een script eigenlijk?

Een script is vergelijkbaar met een recept: het bevat de instructies om iets lekkers te koken, maar niet de ingredienten of het resultaat zelf. Deze instructies staan in zogeheten "plain text" formaat. Dit betekent dat de tekst geen vetgedrukte woorden, cursief, onderstrepingen, kleuren of speciale lettertypes bevat. Plain tekst kan data beschrijven (bijvoorbeeld met elke regel een observatie), of in het geval van een R-script een set van instructies. Elke regel bevat hoogstens 1 instructie, en deze code kan regel-per-regel worden uitgevoerd, of in één keer.

Kopieer voor het volgende onderdeel de volgende code naar je eigen script:

```
1+2
1+2-3
1+2-3*4/5
```

## 2.2 Script en console

In Rstudio kun je op meerdere manieren regels code uitvoeren. Om de code uit je script regel-per-regel uit te voeren, klik je de bovenste regel aan (niet selecteren, maar gewoon ergens in de regel klikken), en gebruik je `Run > Run Selected Line(s)` (de sneltoets hiervoor is `Ctrl/Cmd+Enter`, wat heel handig is om in je vingers te krijgen!). Nu je de eerste regel hebt uitgevoerd, zie je in de console (onder het script) het antwoord: `[1] 3`. De `[1]` geeft aan dat dit het eerste (enige) waarde is die de berekening heeft gegeven. Later zul je zien dat een berekening ook meerdere waarden terug kan geven. Je kan ook het hele script in één keer uitvoeren door `Run > Run all` te kiezen.

De bovenstaande uitleg geeft aan hoe je het script (het "recept") uit kan voeren, maar wat nu als je even snel iets wil uitproberen voordat je het recept helemaal overhoop gooit? Hiervoor kun je direct in de console typen. Als je bijvoorbeeld de berekening `80/250` in de console typt, heb je gauw

een antwoord op je vraag maar is je script niet aangepast. De console is dus handig om even wat uit te proberen, maar let op: deze code wordt niet opgeslagen als je het script opslaat!

## 2.3 Rekenen met R

Zoals je nu misschien begrijpt, is R een geweldig krachtige rekenmachine! In plaats van alles handmatig uit te rekenen of in je grafische rekenmachine in te typen, kunnen we onze eigen R-scripts maken die berekeningen gemakkelijk en reproduceerbaar maken. Om dit nog makkelijker te maken, kunnen we gebruik maken van variabelen:

```
# a, b, en c waardes toekennen (assigning variables)
a <- 1
b <- 3
c <- 2.5

# rekenen met a, b, en c
a+b-c # a plus b min c
a*b  # a maal b
a/b  # a gedeeld door b
b*b  # b maal b
b^2  # b kwadraad
sqrt(b) # wortel (sqaure root) van b


# andere manieren om variabelen te definieren
3 -> d
e = 2.5

# een uitgebreider sommetje uitrekenen
d/e+d*(c+d)

# deze som kunnen we ook weer opslaan in een variabele
uitkomst <- d/e+d*(c+d)
```

## 2.4 Comments

In de bovenstaande code zie je meerdere regels met een hekje (#) ervoor. Alle tekst na een hekje wordt door R genegeerd, zodat we onze code opmerkingen kunnen plaatsen. Deze opmerkingen heten in het Engels *comments*.

## 2.5 De "Environment"

Variabelen die je definieert worden opgeslagen in de **Environment**. Waar het script een soort "recept" is, is de **Environment** meer vergelijkbaar met de keukentafel: hier worden alle ingeladen gegevens, berekeningen, en grafieken opgeslagen terwijl je aan je gerecht werkt. De environment helpt je om een overzicht te houden van wat er allemaal gebeurt, en blijft behouden totdat je Rstudio afsluit. Let op: ook de stapjes die je met de console hebt gedaan komen op deze keukentafel terecht, maar als je wil dat iemand anders het gerecht kan maken moet je zorgen dat ook het recept klopt. Dit recept (het script) is namelijk vaak het enige wat we zullen opslaan. Zo kunnen we de volgende keer ons gerecht opnieuw maken, of ons gerecht met andere delen. En let op: als je de volgende keer Rstudio opstart, begin je opnieuw met koken, en begint je keukentafel dus weer leeg!

## 2.6 Opgave onderdeel 1

Om te testen of je de informatie uit onderdeel 1 hebt begrepen staat hier een simpele opgave. Lees de code hierboven nog eens door, en beantwoord de volgende vragen:

1. Definieer a=2, en reken met R uit wat $a^3$ is.
2. Definieer b=9.5 en reken uit wat $b^{19}$ is (let op: dit hoeft geen lange regel te zijn)
3. Stel je zet eerst 3 stappen naar links, dan 4 stappen naar boven, en loopt vervolgens terug naar je beginpositie. Schrijf een script met R die uitrekent hoe ver je in totaal hebt gelopen. Zorg dat de uitkomst van je berekening **niet** in de environment staat, maar slechts in de console wordt weergegeven.

4. Bereken nu uit hoever je gelopen zou zijn als je in plaats hiervan 5 stappen naar links en 6 stappen naar boven had gezet. Zorg dit keer dat de uitkomst **wel** in de environment staat on de de naam “afstand”.
5. Voeg de volgende regel toe aan je script:
   `print(paste('Je hebt', afstand, 'stappen gelopen'))`. Voer je hele script uit.
6. Sla je script op!

```
# Dit is simpelweg de stelling van pythagoras:
a <- 5
b <- 6
c <- sqrt(a^2+b^2)
afstand <-a+b+c
print(paste('Je hebt', afstand, 'stappen gelopen'))
```

# 3 Dag 2: programmeren met R

## 3.1 Typen variabelen

We hebben in het eerste onderdeel al een beetje kennis gemaakt met variabelen. Maar je kan in variabelen niet alleen nummers opslaan, maar ook een heleboel andere type gegevens. De meest belangrijke variabelen zijn:

- *Numerics* (`1`, `2`, `3`, maar ook `2.158196` zijn *numeric*)
- *Logicals* (`TRUE` en `FALSE`)
- *Strings* (`"a"`, `"Hoi"`, of `"bananen zijn krom"`)

## 3.2 Geen appels en peren vergelijken

In R kun je berekeningen uitvoeren, maar je kan natuurlijk geen appels en peren vergelijken. Bekijk bijvoorbeeld devolgende code:

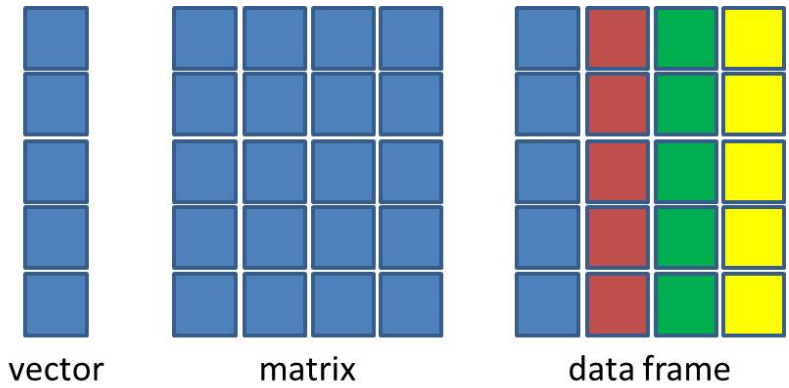
```
appels <- 3
peren <- "groen"
appels + peren
```

Als je de bovenstaande code probeert uit te voeren, geeft R een foutmelding: `Error in appels + peren : non-numeric argument to binary operator`. Dat is maar goed ook, want hoewel dit een overduidelijk onzinnige som is, kan het ook zijn dat je met de beste intenties een foutje maakt!

## 3.3 Data structuren

Het is vaak heel handig om je variabelen in lijsten of tabellen op te slaan. Dergelijke lijsten of tabellen worden meestal *data structures* genoemd. De meest belangrijke data structures in R zijn:

- *Vectors* (lijst van elementen van hetzelfde type)
- *Macrices* (tabel van elementen van hetzelfde type)
- *Dataframes* (tabel waarin elke kolom een ander type data kan zijn)




Figuur 3.1: datastructs

*(figuur is niet van mij, misschien even namaken)*

### 3.3.1 Vectoren

Om een vector te maken, kunnen we deze handmatig intypen:

```
mijnvector <- c(1,2,0.1,-5)
```

Maar handmatig getallen invoeren is niet altijd handig. Gelukkig kun je met R een boel lijstjes maken op een snellere manier:

```
1:4                 # 1, 2, 3, 4
5:2                 # 5, 4, 3, 2
rep(2,4)            # 2, 2, 2, 2
seq(0,4.5,1.5)      # 0.0, 1.5, 3.0, 4.5
sample(1:6,4)       # 4, 6, 3, 5 (iedere keer anders!)
sample(letters,4)   # "f", "j", "o", "y" (iedere keer anders!)
runif(2)            # 0.9965360 0.8978062 (iedere keer anders!)
```

De laatste vijf regels van het bovenstaande script zijn voorbeelden van "functies". Een functie is een herhaalbaar blokje code die je aanroept met de naam (rep, seq, sample), gevolgd door haakjes met daartussen eventuele *argumenten*. De functie zet deze argumenten om in een zogeheete return-waarde, wat bijvoorbeeld een getal of een lijst getallen is. Je leert meer over functies in het onderdeel Paragraaf **??**.

Je kan met vectoren ook rekenen. Zo kan je bijvoorbeeld twee vectoren bij elkaar optellen, of één en hetzelfde getal bij alle elementen van een vector optellen:

```
# Rekenen met vectoren
x <- 1:3            # 1, 2, 3
x + 10              # 11 12 13
y <- c(1, 10, 4)    # 1 10 4
x + y               # 2 12  7
x * y               # 1 20 12
x + y / 100         # 1.01 2.10 3.04
(x + y) / 100       # 0.02 0.12 0.07
```

In de bovenstaande code zie je een subtiel verschil tussen de laatste twee regels. Net als bij wiskunde gaan delen en vermedigvuldigen altijd voor, dus moet je soms aangeven in welke volgorde je de berekeninng wil uitvoeren!

### 3.3.2 Matrices

Matrices in R kun je maken door de functie `matrix` aan te roepen, die je waarden van de matrix en het aantal rijen als argumenten geeft:

```r
matrix(c(1, 2, 3, 4,5, 6, 7, 8, 9), nrow=3)
matrix(c(1, 2, 3, 4,5, 6, 7, 8, 9), nrow=3, byrow=T)
```

Zoals je ziet zijn er geen comments geplaatst bij deze twee regels. Voer de bovenstaande code zelf uit, en kijk naar de verschillende uitkomsten van de eerste en de tweede regel. Je kan de matrix ook vullen met andere waarden:

```r
kansen_matrix <- matrix(runif(9),nrow=3) # willekeurige getallen tussen 0 e
percentage_matrix <- kansen_matrix*100   # de kansen matrix keer 100, dus p
letter_matrix <- matrix(sample(letters,9),nrow=3) # willekeurige letters
```

### 3.3.3 Data frames

De laatste (en misschien wel belangrijkste) data structuur in R is de data frame. In een dataframe bevat elke kolom een ander gegeven (naam, leeftijd, haarkleur), en is elke rij een apart observatie. Een dergelijke structuur is heel handig om wetenschappelijke data te verzamelen. Ook deze kun je weer handmatig invoeren:

```r
biologen <- data.frame(naam=c("Lina","Omar","Bram"),
                       leeftijd=c(19,18,20),
                       haarkleur=c("Blauw","Groen", "Paars"))
```

Echter is het natuurlijk onzinnig om handmatig data frames in te typen in R. In plaats daarvan kun je grote datasets vaak opslaan of downloaden in een groot tekstbestand, bijvooreeld in *comma-seperated-value* (csv) bestanden. Als je een dergelijk tekstbestand hebt, kun je deze inlezen met de functie `read.table` of `read.csv`:

```r
covid_data <- read.csv("COVID_testen_Utrecht.csv") #deze file moet wel in j
head(covid_data) # bekijk de eerste 6 rijen
View(covid_data) # bekijk de hele tabel in Rstudio
```

## 3.4 Files, paden, en de 'working directory'

Om (zoals hierboven) een bestand in te kunnen lezen, moet Rstudio wel weten waar het bestand staat. Standaard zoekt Rstudio vaak in de home directory van je besturingsysteem (`/Users/Naam` in MacOs, `C:/Users/UserName/Documents` in Windows). De directory vanuit waar Rstudio naar het bestand zoekt, heet de *Working Directory*. Vaak is het handig om je Working Directory aan te passen naar dezelfde map waar je het script hebt opgeslagen, zodat je alle data, scripts, en grafieken een beetje bij elkaar kan houden. Dit kan je doen door te navigeren naar `Session > Set Working Directory > To Source File Location`.

## 3.5 Indexing

Indexing is het selecteren van specifieke waarden uit een vector, matrix, of dataframe. Dit doe je door de positie (index) van deze elementen tussen vierkante haakjes aan te geven:

```
jaartallen <- 2009:2024          # Jaren dat ik biologie "studeer"
jaartallen[6:9]                  # 2014 (6e jaar), 2015 (7e jaar)

m <- matrix(runif(9),nrow=3)     # Willekeurige getallen in een matrix
m[1,2]    # rij 1, kolom 2
m[1,]     # rij 1 volledig
m[,2]     # kolom 2 volledig
m[1:2,3]  # van rij 1 t/m 2, alleen de 3e kolom

biologen <- data.frame(naam=c("Lina","Omar","Bram"),
                       leeftijd=c(19,26,36),
                       haarkleur=c("Blauw","Groen", "Paars"))
biologen[1,2] # eerste rij, tweede kolom (leeftijd: 19)
```

Omdat in data frame de kolommen namen hebben, kun je ook deze gebruiken om specifieke datapunten te selecteren:

```r
biologen[1,"leeftijd"] #identiek aan biologen[1,2], maar duidelijker wat je
```

Je kunt met dataframes ook met het dollar-teken een gehele kolom opvragen, een kolom bewerken, of een kolom toevoegen:

```r
biologen$leeftijd
biologen$leeftijd <- biologen$leeftijd + 2  # Alle biolgen zijn 1 jaar oude
biologen$rol <- c("student","assistent","docent") # Een nieuwe kolom genaam
```

# 4 Functies, condities, en loops

## 4.1 Functies en help-functie

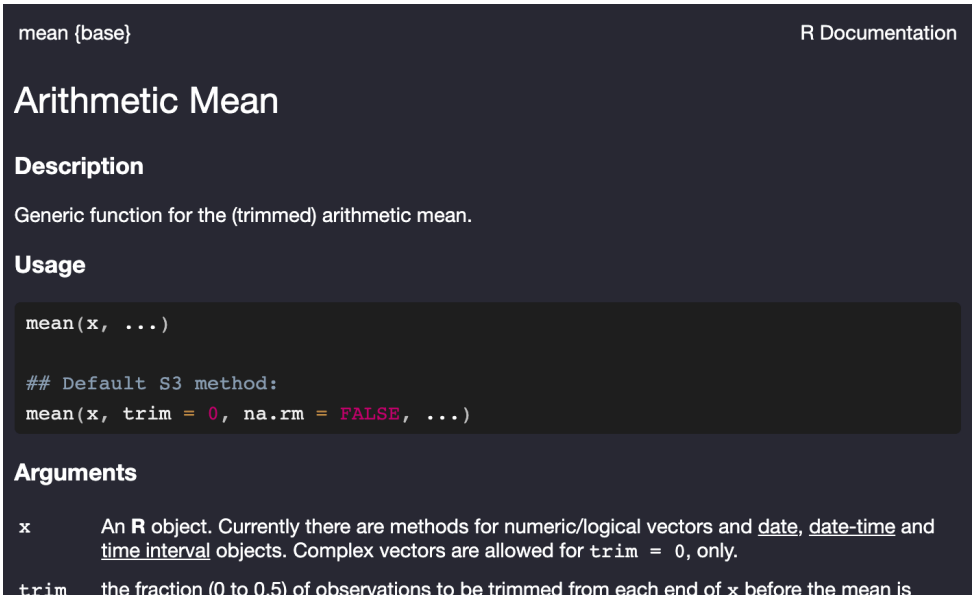
In het onderdeel Paragraaf **??** heb je al een aantal voorbeelden gezien van functies, herhaalbare blokjes code die je gemakkelijk kan aanroepen. Een functies geef je waarden door (deze waarden heten *argumenten*), en geeft één of meerdere waarden terug (deze waarde heet de *return*-waarde). Er bestaan standaard in R een heleboel functies, waarvan je een aantal in deze cursus nog veel zal tegenkomen:

- `read.csv`, leest comma-separated-values bestand in als dataframe
- `print` of `cat`, geef iets (variabelen of tekst) weer in de console
- `plot`, om data te plotten in een grafiek
- `length` geeft de lengte van een vector
- `mean`, gemiddelde van een vector
- `sd`, standaarddeviatie van een vector
- `sum`, som van een vector
- `min` geeft de kleinste waarde in een vector
- `max` geeft de grootste waarde in een vector
- `sort`, sorteert een vector

Er zijn nog veel meer standaard functies in R. Als je graag meer wil eten over hoe de functie werkt, kun je de naam van de functie typen met een `?` ervoor:

```
?mean
```

Dit geeft extra uitleg voor deze functie in het “Help” venster:




Figuur 4.1: Elephant

## 4.2 Zelf functies maken

Het handige aan functies is dat je ze ook zelf kan maken, als je bijvoorbeeld iets heel specifieks wil doen. Je moet hiervoor aangeven wat de argumenten zijn, welke berekening op basis van deze argumenten wordt uitgevoerd, en wat er door de functie wordt teruggegeven (de return-waarde). Hier is een voorbeeld:

```
# Definitie van de functie
som_kwadraten <- function(a, b) {
  resultaat <- a^2 + b^2
  return(resultaat)
}

# Voorbeeld gebruik van de functie
som_kwadraten(1,2)            # 5
som_kwadraten(3, 4)           # 25
som_kwadraten(0.5, 0.1)       # 0.26
```

## 4.3 Condities

Vaak is het handig om in een script te controleren of aan een bepaalde voorwaarde wordt voldaan. Hiervoor kun je `if`, `else`, en `else if` gebruiken:

```
leeftijd_student <- sample(16:19,1)
volwassen_leeftijd <- 18
if(leeftijd_student >= volwassen_leeftijd){
  print("Student is volwassen")
} else{
  print("Student is niet volwassen")
}
```

Om voorwaarden te controleren, gebruik je zoheten **conditionals**, die teruggeven of iets waar (`TRUE`) of niet waar (`FALSE`) is. Hierboven gebruiken we `>=` (groter dan of gelijk aan) om te controleren of de student minstens 18 jaar oud is. Veelgebruikte andere conditionals zijn:

- `a == b` : TRUE als a gelijk is aan b
- `a != b` : TRUE als a niet gelijk is aan b
- `a > b` : TRUE als a is groter is dan b
- `a < b` : TRUE als a is kleiner is dan b
- `a >= b` : TRUE als a is groter is dan of gelijk is aan b
- `a <= b` : TRUE als a is kleiner is dan of gelijk is aan b

Daarnaast kun je eventueel meerdere condities combineren:

- `a==b & b!=c` : TRUE als a gelijk is aan b en b **niet** gelijk is aan c
- `a<b | b>c` : TRUE als a kleiner is dan b en b groter is dan c

Als laatste kun je nog veel meer combinaties controleren door `if-else` elkaar op te laten volgen en te nesten. Als we bijvoorbeeld twee dobbelsteenworpen simuleren, kunnen we precies controleren welke dobbelsteen hoger is dan 4, en een voor ieder scenario een unieke uitkomst maken:

```r
d1 <- sample(1:6,1) # dobbelsteen 1
d2 <- sample(1:6,1) # dobbelsteen 2

if (d1 > 4) {
  if (d2 > 4) {
    print("d1 en d2 zijn hoger dan 4")
  } else {
    print("d1 is hoger dan 4, maar d2 niet")
  }
} else if(d2 > 4) {
  print("d1 is niet hoger dan 4, maar d2 wel")
}
```

Let op: als je de bovenstaande code goed leest, zie je dat er helemaal **niets** gebeurt als geen van beide dobbelstenen groter dan 4 is.

## 4.4 Loops

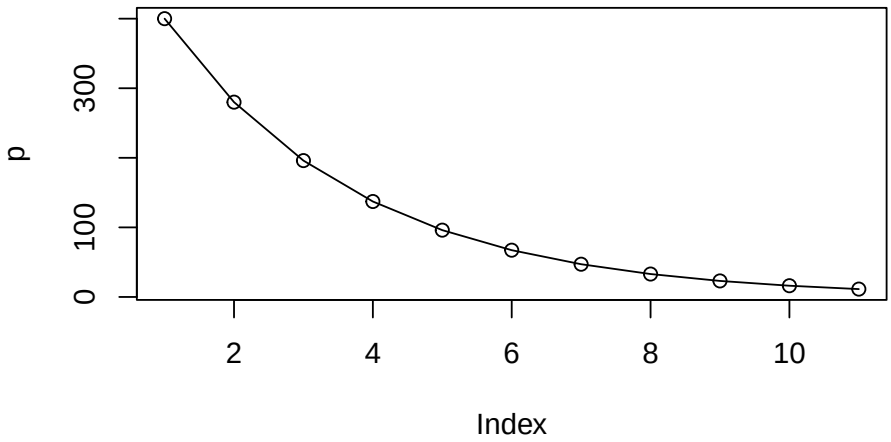
Hoewel functies het makkelijker maken om dezelfde berekening meerdere keren uit te voeren, wat nu als je een functie 100 keer wil aanroepen? Hebben we dan 100 regels nodig? Het antwoord is nee, want elke programmeertaal heeft manieren om stukken code te herhalen. In R bestaan veel verschillende manieren om dit te doen (`repeat`, `apply`, `while`), maar in deze introductie maken we vooral kennis met de meest-gebruikte loop: de for-loop.

Laten we bijvoorbeeld de exponentiele afname van paracetamol in het bloed beschrijven met een simpele simulatie. Iemand neemt 400 milligram paracetamol in, wat ieder uur (ongeveer) met 70% afneemt:

```r
p <- 400 # hoeveelheid paracetamol
d <- 0.7 # afname (d van decay)
# Herhaal iets 10 keer, waarbij 'i' steeds met 1 toeneemt
for(i in 1:10){
  p <- p*d
}
cat('Na 10 uur is er',p,'milligram paracetamol in het bloed') # 11.299 mg
```

Wellicht willen we graag bijhouden hoeveel paracetamol er in elke stap aanwezig was, zodat we dat na onze loop kunnen plotten. Om dit te doen, kunnen we de `i` als index gebruiken om een vector te vullen:

```
p <- 400 # hoeveelheid paracetamol
d <- 0.7 # afname (d van decay)
# Herhaal iets 10 keer, waarbij 'i' steeds met 1 toeneemt
for(i in 1:10){
  p[i+1] <- p[i]*d
}
plot(p, type='o')
```



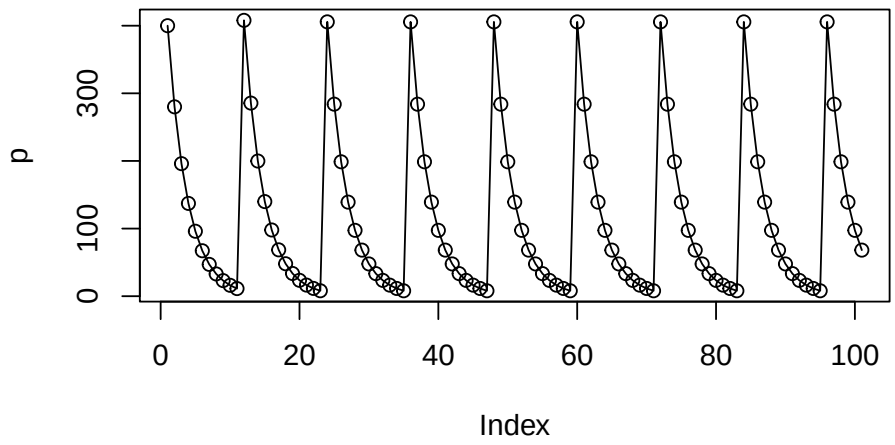

Omdat dit voorbeeld om simpele exponentiele afname gaat, hadden we de bovenstaande berekening ook "in één keer" kunnen uitrekenen door $\$p \cdot d^{10}\$$ uit te rekenen. Laten we dat even controleren:

```
p <- 400 # hoeveelheid paracetamol
d <- 0.7 # afname (d van decay)
p*d^10 # 11.299 mg
```

Dit antwoord komt inderdaad overeen met onze berekening met de `for` loop. Hoewel deze loop dus misschien wat onnodig lijkt, hebben we niet altijd zo'n simpele formule beschikbaar. Bovendien willen we wellicht iets veel specifiekers simuleren, bijvoorbeeld dat de persoon elke 12 uur opnieuw paracetamol inneemt:

```
p <- 400 # hoeveelheid paracetamol
d <- 0.7 # afname (d van decay)

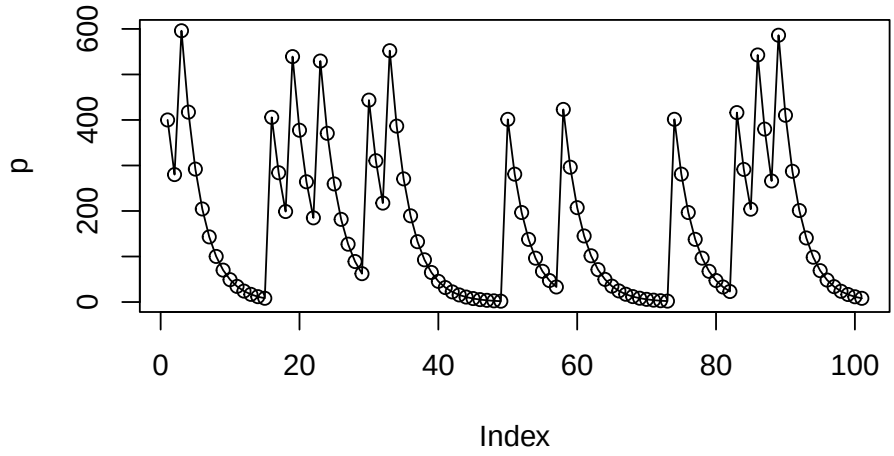
for(i in 1:100){
  if(i%%12==0){ # is i precies deelbaar door 12?
    p[i] <- p[i] + 400
  }
  p[i+1] <- p[i]*d
}
plot(p, type='o')
```




Of dat de persoon met 10% kans elk uur een paracetamol inneemt:

```r
p <- 400 # hoeveelheid paracetamol
d <- 0.7 # afname (d van decay)

for(i in 1:100){
  if(runif(1)<0.1){ # is willekeurig getal (tussen 0 en 1) kleiner dan 0.1?
    p[i] <- p[i] + 400
  }
  p[i+1] <- p[i]*d
}
plot(p, type='o')
```

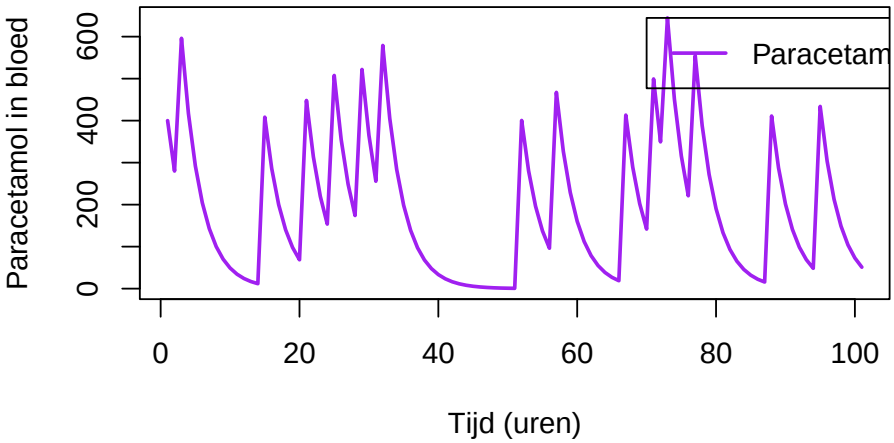


# 5 Grafieken plotten met R

Om de bovenstaande plot van de paracetamol afname iets volledi-
ger en mooier te maken, kunnen we meer argumenten aan de
plotfunctie toevoegen:

```
p <- 400 # hoeveelheid paracetamol
d <- 0.7 # afname (d van decay)

for(i in 1:100){
  if(runif(1)<0.1){ # is willekeurig getal (tussen 0 en 1) kleiner dan 0.1?
    p[i] <- p[i] + 400
  }
  p[i+1] <- p[i]*d
}
plot(p, type='l', lwd=2, col="purple", xlab="Tijd (uren)", ylab="Paracetamol
legend(70, max(p), legend=c("Paracetamol"), col=c("purple"),lty=1,lwd=2)
```




Er zijn binnen R nog veel meer manieren om data te plotten, zoals bijvoorbeeld de `boxplot` die je bij het onderdeel statistiek zeker nog tegen zal komen. Daarnaast zijn er in R veel `packages` beschikbaar kunnen helpen bij het maken van mooie grafieken, zoals ggplot maar daar gaan we in deze cursus niet dieper op in.

### 5.0.1 Opgaven deel 2: TODO

1. Neem bovenstaande script over in R (tip: je kan de hele code in 1 keer kopiëren met het knopje rechtsboven)
2. Voeg aan het einde van een script een extra regel toe, waar je de variabele `e` in `2` verandert.
3. Voer het script uit. De waarde van de uitkomst-variabele is niet verandert. Leg met de kook-analogie uit waarom niet.

# 6 Wat moet ik weten over R?

1. Wat is het verschil tussen een matrix in een data frame
   A: matrix kan alleen dezelfde waarden bevatten, data frame kan (per kolom) verschillende type data bevatten.

2. `x <- seq(1,10,by=0.5)`

   Wat geeft x[2] ?
   opties (**1**, 0.5, 2, NA)

3. `x <- 10:1`

   Wat geeft x[1] ?
   antwoord = 10

4. x <- runif(10)

5. Wat geeft x[11]
   antwoord = NA

6. Beschouw hetvolgende data frame:

   ```
   df <- data.frame(   Naam = c("Jaap", "Alina", "Xin"),
   Minuten = c(12, 19, 11),   Score = c(85, 90, 95)
   )
   ```

   Stel we willen een nieuwe kolom toevoegen die de score per minuut geeft, wat moeten we daarvoor doen?
   A (goede): `df$score_per_minuut <- df$score/df$minutenA` (fout): `df$score <- df$score/df$minutenA` (fout): `df[score_per_minuut] <- df$score/df$minutenA` (fout): `df + score_per_minuut <- df$score/df$minutenA (fout): df$score_per_minuut <- "score per minuut"`

7. Een student leest de volgende csv in van dieren in het asiel onder de naam “dieren”:
   (iets met comma-separated values)

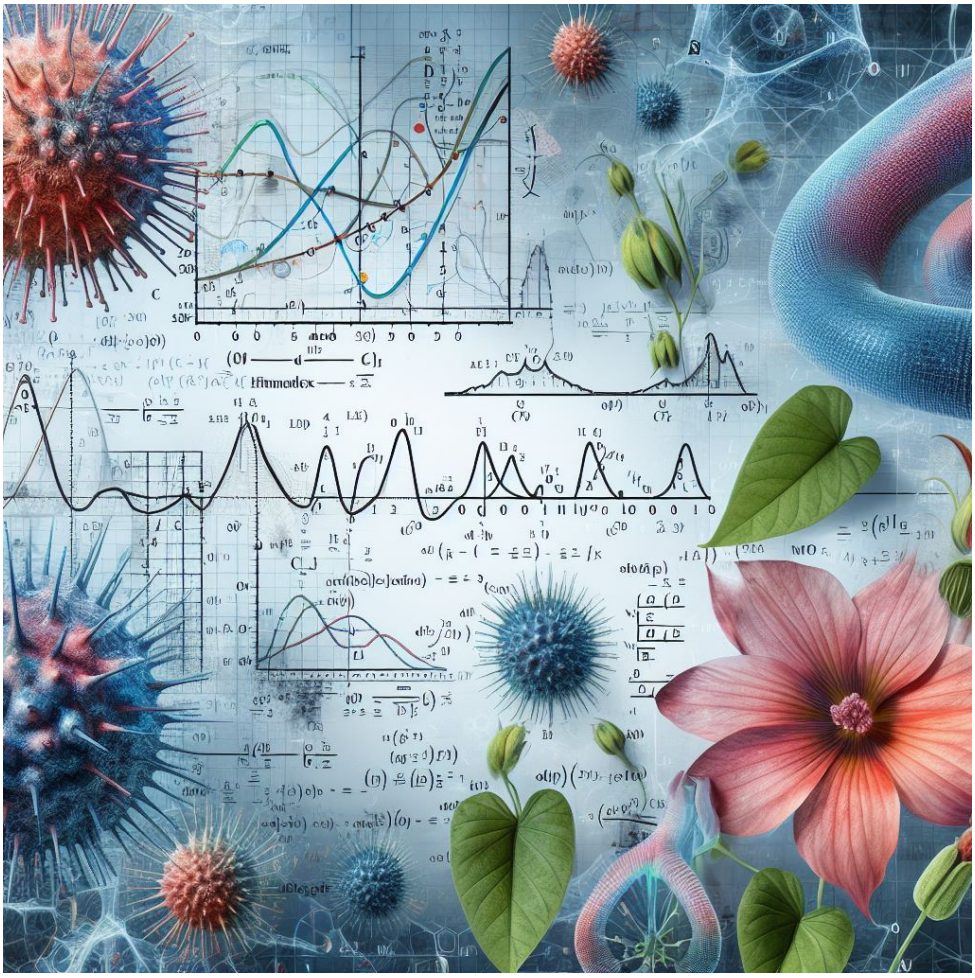
   Naam, Leeftijd, Vachtkleur, Temperament
   Joey, 8, zwart, rustig

   Niem, 2, wit, energiek
   Bob, 4, bruin, vriendelijk
   Saartje, 2, gevlekt, verlegen

   dieren[2,3:4] geeft:
   **“wit” en “energiek”**
   ”zwart” en “rustig”
   4 en 2
   4 en “bruin”
   2 en “wit”
   “verlegen” en “gevlekt”
   8 en “zwart”
8. Een for-loop met wat uitkmost afhankelijk van een if/else statement.
9.

# Deel III

# Basiswiskunde



Figuur 6.1: Wiskunde in de biologie; illustratie gegenereerd door AI. (DALL-E 3, maart 2024)

# 7 Breuken

# 8 Exponenten

# 9 Logaritmen

# 10 Vergelijkingen oplossen

# 11 Limieten en asymptoten

# 12 Schetsen

# 13 Afgeleiden